# Московский авиационный институт
# (национальный исследовательский университет)

## Факультет информационных технологий и прикладной математики

## Кафедра вычислительной математики и программирования


**Лабораторная работа №3 по курсу «Криптография»**


| | |
|---|---|
| Студент: | М. О. Живых |
| Преподаватель: | А. В. Борисов |
| Группа: | М8О-307Б |
| Дата: | |
| Оценка: | |
| Подпись: | |



**Москва, 2020**

# Лабораторная работа №3

**Задача:**

1. Строку в которой записано своё ФИО подать навход в хеш-функцию ГОСТ Р 34.11-2012 (Стрибог). Младшие 4 бита выхода интерпретировать как число, которое в дальнейшем будет номером варианта.

2. Программно реализовать алгоритм функции хеширования. Алгоритм содержит в себе несколько раундов.

3. Модифицировать оригинальный алгоритм таким образом, чтобы количество раундов было настраиваемым параметром программы. В этом случае новый алгоритм не будет являться стандартом, но будет интересен для исследования.

4. Применить подходы дифференциального криптоанализа к алгоритму с разным числом раундов.

5. Построить график зависимости количества раундов и возможности различения отдельных бит при различном количестве раундов.

**Вариант B:** MD4.

# 1 Расчёт варианта

Для расчета варианта я написал программу на языке *Python*, которая при помощи сторонней библиотеки **pygost** вычисляет хэш переданной ей строки (в моем случае «Бронников Максим Андреевич»), после чего при помощи операции «побитового И» отделяет последние 4 бита от хэша и выводит их в удобной для восприятия форме.

*Исходный код программы:*

```python
from pygost import gost34112012256
import sys


def number_from_str(family):
    if not isinstance(family, str):
        return -1
    # Working only with .encode() method, else TypeError
    last = gost34112012256.new(family.encode()).digest()[-1]
    last &= 15
    if last >= 10:
        return chr(ord('A') + last - 10)
    return str(last)

if __name__ == "__main__":
    family = "Бронников МаксимАндреевич"

    if len(sys.argv) == 2 and sys.argv[1] == "-i":
        family = input("Введите вашеФИО: ")
    elif len(sys.argv) > 1:
        raise ValueError("Wrong args")

    print("Вариант для" + family + ":", number_from_str(family))
```

*Результат вычисления:*

```
"C: Era38-32.exe" "C:/Users/Tyruru Era/PycharmProjects/Test/Main.py"
Вариант для Живых Максим Олегович: B

Process finished with exit code 0
```

## 2 Описание алгоритма

MD4 ($MessageDigest4$) – хеш-функция, разработанная профессором Массачусетского технологического института Рональдом Ривестом в 1990 году, и впервые описанная в RFC 1186. Для произвольного входного сообщения функция генерирует 128-разрядное хеш-значение, называемое дайджестом сообщения. Этот алгоритм используется в протоколе аутентификации MS-CHAP, разработанном корпорацией Майкрософт для выполнения процедур проверки подлинности удаленных рабочих станций Windows. Является предшественником MD5.

Алгоритм MD4
Предполагается, что на вход подано сообщение, состоящее из $b$ бит, хеш которого нам предстоит вычислить. Здесь $b$ — произвольное неотрицательное целое число; оно может быть нулем, не обязано быть кратным восьми, и может быть сколь угодно большим. Запишем сообщение побитово, в виде: $m_0 m_1 ... m_{b-1}$. Ниже приведены 5 шагов, используемые для вычисления хеша сообщения.

Шаг 1. Добавление недостающих битов.
Сообщение расширяется так, чтобы его длина в битах по модулю 512 равнялась 448. Таким образом, в результате расширения, сообщению недостает 64 бита до длины, кратной 512 битам. Расширение производится всегда, даже если сообщение изначально имеет нужную длину. Расширение производится следующим образом: один бит, равный 1, добавляется к сообщению, а затем добавляются биты, равные 0, до тех пор, пока длина сообщения не станет равной 448 по модулю 512. В итоге, к сообщению добавляется как минимум 1 бит, и как максимум 512.

Шаг 2. Добавление длины сообщения.
64-битное представление $b$ (длины сообщения перед добавлением набивочных битов) добавляется к результату предыдущего шага. В маловероятном случае, когда $b$ больше, чем $2^{64}$, используются только 64 младших бита. Эти биты добавляются в виде двух 32-битных слов, и первым добавляется слово, содержащее младшие разряды. На этом этапе (после добавления битов и длины сообщения) мы получаем сообщение длиной кратной 512 битам. Это эквивалентно тому, что это сообщение имеет длину, кратную 16-ти 32-битным словам. Каждое 32-битное слово содержит четыре 8-битных, но следуют они не подряд, а наоборот (например, из восьми 8-битных слов ($abcdefgh$) мы получаем два 32-битных слова ($dcbahgfe$)). Пусть $M[0...N1]$ означает массив слов получившегося сообщения (здесь $N$ кратно 16).

Шаг 3. Инициализация MD-буфера.
Для вычисления хеша сообщения используется буфер, состоящий из 4 слов (32-

битных регистров): $(A, B, C, D)$. Эти регистры инициализируются следующими шестнадцатеричными числами (младшие байты сначала):

$wordA$ : 67 45 23 01
$wordB$ : ef cd ab 89
$wordC$ : 98 ba dc fe
$wordD$ : 10 32 54 76

Шаг 4. Обработка сообщения блоками по 16 слов.
Для начала определим три вспомогательные функции, каждая из которых получает на вход три 32-битных слова, и по ним вычисляет одно 32-битное слово.

$F(X, Y, Z) = XY \vee \neg XZ$
$G(X, Y, Z) = XY \vee XZ \vee YZ$
$H(X, Y, Z) = X \oplus Y \oplus Z$

На каждую битовую позицию $F$ действует как условное выражение: если $X$, то $Y$; иначе $Z$. Функция $F$ могла бы быть определена с использованием + вместо , поскольку $XY$ и $\neg$ $XZ$ не могут равняться 1 одновременно. $G$ действует на каждую битовую позицию как функция максимального значения: если по крайней мере в двух словах из $X, Y, Z$ соответствующие биты равны 1, то $G$ выдаст 1 в этом бите, а иначе $G$ выдаст бит, равный 0. Интересно отметить, что если биты $X, Y$ и $Z$ статистически независимы, то биты $F(X, Y, Z)$ и $G(X, Y, Z)$ будут также статистически независимы. Функция $H$ реализует побитовый $xor$, она обладает таким же свойством, как $F$ и $G$.

Замечание. Величина $5A827999$ — шестнадцатеричная 32-битная константа, первые байты — старшие. Она представляет собой квадратный корень из 2. Она же в восьмеричном представлении: 013240474631. Величина $6ED9EBA1$ — шестнадцатеричная 32-битная константа, первые байты — старшие. Она представляет собой квадратный корень из 3. Она же в восьмеричном представлении: 015666365641. Эти данные приведены в книге Кнут, Искусство программирования, издание 1981 года, том 2, стр 660, таблица 2.

Шаг 5. Формирование хеша.
Результат (хеш-функция) получается как ABCD. То есть, мы выписываем 128 бит, начиная с младшего бита A, и заканчивая старшим битом D.

# 3 Реализация алгоритма

Для реализации я использовал язык *C#*.

*Исходный код:*

```
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Text;

static class Md4
{
  private static uint A, B, C, D;
  private static uint[] X;

  private static Func<uint, uint, uint, uint> F = (x, y, z) => (x & y) | (~x & z);
  private static Func<uint, uint, uint, uint> G = (x, y, z) => (x & y) | (x & z) | (y
      & z);
  private static Func<uint, uint, uint, uint> H = (x, y, z) => x ^ y ^ z;

  private static Func<uint, uint, uint> leftRotate = (x, y) => x << (int)y | x >> 32 -
       (int)y;

  private static Func<uint, uint, uint, uint, uint, ushort, uint> ROUND10 = (a, b, c,
      d, x, m) => A = leftRotate((a + F(b, c, d) + x), m);
  private static Func<uint, uint, uint, uint, uint, ushort, uint> ROUND11 = (d, a, b,
      c, x, m) => D = leftRotate((d + F(a, b, c) + x), m);
  private static Func<uint, uint, uint, uint, uint, ushort, uint> ROUND12 = (c, d, a,
      b, x, m) => C = leftRotate((c + F(d, a, b) + x), m);
  private static Func<uint, uint, uint, uint, uint, ushort, uint> ROUND13 = (b, c, d,
      a, x, m) => B = leftRotate((b + F(c, d, a) + x), m);

  private static Func<uint, uint, uint, uint, uint, ushort, uint> ROUND20 = (a, b, c,
      d, x, m) => A = leftRotate((a + G(b, c, d) + x + (uint)0x5a827999), m);
  private static Func<uint, uint, uint, uint, uint, ushort, uint> ROUND21 = (d, a, b,
      c, x, m) => D = leftRotate((d + G(a, b, c) + x + (uint)0x5a827999), m);
  private static Func<uint, uint, uint, uint, uint, ushort, uint> ROUND22 = (c, d, a,
      b, x, m) => C = leftRotate((c + G(d, a, b) + x + (uint)0x5a827999), m);
  private static Func<uint, uint, uint, uint, uint, ushort, uint> ROUND23 = (b, c, d,
      a, x, m) => B = leftRotate((b + G(c, d, a) + x + (uint)0x5a827999), m);

  private static Func<uint, uint, uint, uint, uint, ushort, uint> ROUND30 = (a, b, c,
      d, x, m) => A = leftRotate((a + H(b, c, d) + x + (uint)0x6ed9eba1), m);
  private static Func<uint, uint, uint, uint, uint, ushort, uint> ROUND31 = (d, a, b,
      c, x, m) => D = leftRotate((d + H(a, b, c) + x + (uint)0x6ed9eba1), m);
  private static Func<uint, uint, uint, uint, uint, ushort, uint> ROUND32 = (c, d, a,
      b, x, m) => C = leftRotate((c + H(d, a, b) + x + (uint)0x6ed9eba1), m);
```

```
private static Func<uint, uint, uint, uint, uint, ushort, uint> ROUND33 = (b, c, d, a, x, m) => B = leftRotate((b + H(c, d, a) + x + (uint)0x6ed9eba1), m);


public static string Md4Hash(string input, int roundsCount)
{
  byte[] bytesRead = Encoding.UTF8.GetBytes(input);

  if (roundsCount < 1 || roundsCount > 3)
    throw new Exception("Rounds error (chose 1, 2 or 3 rounds count)");

  List<byte> bytes = new List<byte>();

  foreach (var by in bytesRead)
  {
    bytes.Add(by);
  }

  uint bitCount = (uint)(bytes.Count) * 8;

  bytes.Add(128);
  while (bytes.Count % 64 != 56)
  {
    bytes.Add(0);
  }

  var uints = new List<uint>();
  for (int i = 0; i + 3 < bytes.Count; i += 4)
  {
    uints.Add(bytes[i] | (uint)bytes[i + 1] << 8 | (uint)bytes[i + 2] << 16 | (uint)bytes[i + 3] << 24);
  }

  uints.Add(bitCount);
  uints.Add(0);

  A = 0x67452301;
  B = 0xefcdab89;
  C = 0x98badcfe;
  D = 0x10325476;


  for (int i = 0; i < uints.Count() / 16; i++)
  {
    X = new uint[16];
    for (int j = 0; j < 16; j++)
    {
      X[j] = uints[i * 16 + j];
    }
```

```

    uint AA = A;
    uint BB = B;
    uint CC = C;
    uint DD = D;


    if (roundsCount >= 1)
    {
      ROUND10(A, B, C, D, X[0], 3);
      ROUND11(D, A, B, C, X[1], 7);
      ROUND12(C, D, A, B, X[2], 11);
      ROUND13(B, C, D, A, X[3], 19);
      ROUND10(A, B, C, D, X[4], 3);
      ROUND11(D, A, B, C, X[5], 7);
      ROUND12(C, D, A, B, X[6], 11);
      ROUND13(B, C, D, A, X[7], 19);
      ROUND10(A, B, C, D, X[8], 3);
      ROUND11(D, A, B, C, X[9], 7);
      ROUND12(C, D, A, B, X[10], 11);
      ROUND13(B, C, D, A, X[11], 19);
      ROUND10(A, B, C, D, X[12], 3);
      ROUND11(D, A, B, C, X[13], 7);
      ROUND12(C, D, A, B, X[14], 11);
      ROUND13(B, C, D, A, X[15], 19);
    }

    if (roundsCount >= 2)
    {
      ROUND20(A, B, C, D, X[0], 3);
      ROUND21(D, A, B, C, X[4], 5);
      ROUND22(C, D, A, B, X[8], 9);
      ROUND23(B, C, D, A, X[12], 13);
      ROUND20(A, B, C, D, X[1], 3);
      ROUND21(D, A, B, C, X[5], 5);
      ROUND22(C, D, A, B, X[9], 9);
      ROUND23(B, C, D, A, X[13], 13);
      ROUND20(A, B, C, D, X[2], 3);
      ROUND21(D, A, B, C, X[6], 5);
      ROUND22(C, D, A, B, X[10], 9);
      ROUND23(B, C, D, A, X[14], 13);
      ROUND20(A, B, C, D, X[3], 3);
      ROUND21(D, A, B, C, X[7], 5);
      ROUND22(C, D, A, B, X[11], 9);
      ROUND23(B, C, D, A, X[15], 13);
    }

    if (roundsCount >= 3)
    {
```

```
        ROUND30(A, B, C, D, X[0], 3);
        ROUND31(D, A, B, C, X[8], 9);
        ROUND32(C, D, A, B, X[4], 11);
        ROUND33(B, C, D, A, X[12], 15);
        ROUND30(A, B, C, D, X[2], 3);
        ROUND31(D, A, B, C, X[10], 9);
        ROUND32(C, D, A, B, X[6], 11);
        ROUND33(B, C, D, A, X[14], 15);
        ROUND30(A, B, C, D, X[1], 3);
        ROUND31(D, A, B, C, X[9], 9);
        ROUND32(C, D, A, B, X[5], 11);
        ROUND33(B, C, D, A, X[13], 15);
        ROUND30(A, B, C, D, X[3], 3);
        ROUND31(D, A, B, C, X[11], 9);
        ROUND32(C, D, A, B, X[7], 11);
        ROUND33(B, C, D, A, X[15], 15);
      }


      A += AA;
      B += BB;
      C += CC;
      D += DD;
    }

    byte[] exportBytes = new[] { A, B, C, D }.SelectMany(BitConverter.GetBytes).ToArray
        ();
    return BitConverter.ToString(exportBytes).Replace("-", "").ToLower();
  }

  public static string RandomString(int length)
  {
    Random random = new Random();
    const string chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";

    return new string(Enumerable.Repeat(chars, length)
      .Select(s => s[random.Next(s.Length)]).ToArray());
  }

  private static readonly string inputFileName = "in";
  private static readonly string outputFileName = "out";


  static void Main()
  {
    int choice;
    do
    {
      Console.WriteLine("1) Make random strings in file.txt");
```

```
      Console.WriteLine("2) Make hash from file.txt in this folder");
      Console.WriteLine("3) Differential cryptanalysis of file.txt");
      Console.WriteLine("4) EXIT");
      Console.WriteLine();

      choice = int.Parse(Console.ReadLine());

      switch (choice)
      {
        case 1:
          using (StreamWriter sw = File.CreateText(Path.Combine(Environment.
              CurrentDirectory, inputFileName + ".txt")))
          {
            Random random = new Random();

            Console.WriteLine("Test count:");
            int testCount = int.Parse(Console.ReadLine());

            sw.WriteLine(testCount);
            for (int i = 0; i < testCount; ++i)
            {
              const string chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";

              sw.WriteLine(new string(Enumerable.Repeat(chars, random.Next(1, 100))
                .Select(s => s[random.Next(s.Length)]).ToArray()));
            }
          }
          break;

        case 2:
          for (int roundCount = 1; roundCount <= 3; ++roundCount)
          {
            using (var sr = new StreamReader(Path.Combine(Environment.CurrentDirectory,
                 inputFileName + ".txt")))
            {
              using (var sw = File.CreateText(Path.Combine(Environment.CurrentDirectory,
                   outputFileName + roundCount + ".txt")))
              {
                int testCount = int.Parse(sr.ReadLine());
                for (int i = 0; i < testCount; ++i)
                {
                  sw.WriteLine(Md4Hash(sr.ReadLine(), roundCount));
                }
              }
            }
          }
          break;

        case 3:
```

```
        for (int roundCount = 1; roundCount <= 3; ++roundCount)
        {
          using (var sr = new StreamReader(Path.Combine(Environment.CurrentDirectory,
               inputFileName + ".txt")))
          {
            int testCount = int.Parse(sr.ReadLine());
            byte deltaX = 1;
            double result = 0;
            for (int t = 0; t < testCount; ++t)
            {
              string str = sr.ReadLine();

              byte[] strBytes = Encoding.UTF8.GetBytes(str);
              strBytes[strBytes.Length - 1] ^= deltaX;
              string x = Encoding.Default.GetString(strBytes);

              string strHash = Md4Hash(str, roundCount);
              string xHash = Md4Hash(x, roundCount);

              byte[] strHashBytes = Encoding.UTF8.GetBytes(strHash);
              byte[] xHashBytes = Encoding.UTF8.GetBytes(xHash);


              for (int i = 0; i < strHashBytes.Length; ++i)
              {
                result += strHashBytes[i] ^ xHashBytes[i];
              }
            }
            result /= testCount;
            Console.WriteLine($"Rounds: {roundCount} Dif bits: {result}");
          }
        }


        break;

      case 4:
        return;

      default:
        Console.WriteLine("Bad index");
        break;
    }
  } while (true);
 }
}
```

*Демонстрация работы алгоритма:*

input: "The quick brown fox jumps over the lazy dog"
output: 1bee69a46ba811185c194762abaeae90

input: "The quick brown fox jumps over the lazy cog"
output: b86e130ce7028da59e672d56ad0113df

input: ""
output: 31d6cfe0d16ae931b73c59d7e0c089c0

input: "Hello world!"
output: eb774d54d6fa3b7fea1e8d6ade0b369d

# 4 Анализ алгоритма

Для анализа работы своего алгоритма я воспользовался методом, который называется «Дифференциальный криптоанализ». Он позволяет оценить качество алгоритма при различном количестве раундов.

При анализе я рассмотрел как изменение количества раундов алгоритма влияет на дифференциальную разность выходного хэша, а именно посчитал количество бит выходного хэша, которое различается для двух текстов с разницей в 1 бит.

Для простоты эксперимента я выбрал дифференциал $\Delta X = 1$ и проводил анализ для количества раундов, варьирующегося от 1 до классических для *Keccak-1600* 24 раундов.

Для этого я генерировал случайным способом 1000000 различных текстов $X_i$, $\forall i : 0 \leq i < n$, и для каждого текста $X_i$ произвел следующую последовательность действий:

1. Сгенерировал 2-ой текст $\overline{X_i} = X_i \oplus \Delta X$, отличающийся от исходного текста $X_i$ лишь 1 битом.

2. Для каждого анализируемого количества раундов $\forall j : \ 1 \leq j \leq 3$:

   - Вычисляем значение хэшэй $Y_i^j = H^j(X_i)$ и $\overline{Y_i^j} = H^j(\overline{X_i})$ при помощи нашей хэш-функции $H^j$ с настраиваемым количеством раундов $j$.

   - Вычисляем их дифференциал $\Delta Y_i^j = Y_i^j \oplus \overline{Y_i^j}$.

   - Подсчитываем количество битов $c_i^j = \sum\limits_{k=0}^{|\Delta Y_i^j|-1} \Delta Y_i^j[k]$, которые различны у двух хэшей $Y_i^j$ и $\overline{Y_i^j}$.

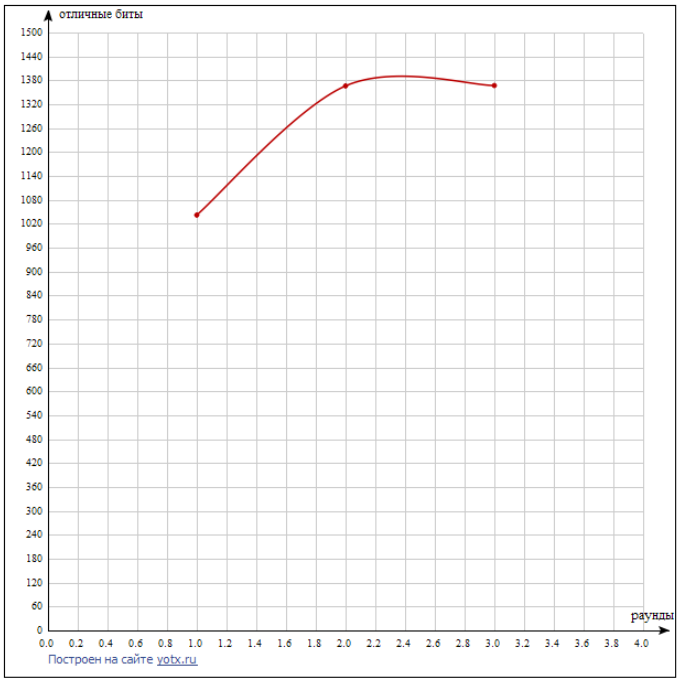
После подсчитаем среднее количество битов $c^j = \dfrac{\sum\limits_{i=0}^{n-1}(c_i^j)}{n}$, которое оказалось различным для каждого анализируемого количества раундов $\forall j : \ 1 \leq j \leq 3$ алгоритма $H^j$.

*Результат вычисления:*

```
1) Make random strings in file.txt
2) Make hash from file.txt in this folder
3) Differential cryptanalysis of file.txt
4) EXIT

1
Test count:
1000000
1) Make random strings in file.txt
2) Make hash from file.txt in this folder
3) Differential cryptanalysis of file.txt
4) EXIT

2
1) Make random strings in file.txt
2) Make hash from file.txt in this folder
3) Differential cryptanalysis of file.txt
4) EXIT

3
Rounds: 1       Dif bits: 1042,821971
Rounds: 2       Dif bits: 1366,258719
Rounds: 3       Dif bits: 1367,158134
```

Здесь первый столбец - количество раундов алгоритма, второй столбец - среднее количество различных бит для $n$ случайных тестовых строк.

И, наконец, построим график зависимости среднего количества различных битов $c^j$ от количества раундов алгоритма $j$:



# 5 Выводы

Выполнив третью лабораторную работу по курсу «Криптография», я получил новые знания о таком понятии, как *Криптографическая хэш-функция*. Мне пришлось узнать и разобраться в том, как они реализуются и даже довелось самостоятельно реализовать один из самых современных алгоритмов хэширования - *MD4*, который лег в основу стандарта *MD5*. Также мне довелось познакомится с новым и несколько пугающим на первый взгляд явлением - дифференциальным криптоанализом и даже применить его простейшие элементы для анализа своего алгоритма.

Несмотря на все сложности и неприятности я не пожалел о том, что мнепришлось столкнуться с этой темой, ведь она оставила у меня огромный багаж новыхзнаний, а также позволила в очередной раз перебороть себя на пути к своей целистать хорошим программистом.

# Список литературы

[1] *Википедия MD4*
URL: `https://ru.wikipedia.org/wiki/MD4`

[2] *GitGub*
URL: `https://github.com/anka3797/md4`$_h ashing$